TEXT CLASSIFICATION

Natural Language Processing

Basic Terminologies

- Dataset, attributes/features, label/class

Outlook	Temp (°F)	Humidity (%)	Windy	Class
sunny	75	70	true	play
sunny	80	90	true	don't play
sunny	85	85	false	don't play
sunny	72	95	false	don't play
sunny	69	70	false	play
overcast	72	90	true	play
overcast	83	78	false	play
overcast	64	65	true	play
overcast	81	75	false	play
rain	71	80	true	don't play
rain	65	70	true	don't play
rain	75	80	false	play
rain	68	80	false	play
rain	70	96	false	play

Basic Terminologies

- Dataset, document, attributes/features, label/class

Text	Lebel
…zany characters and richly applied satire, and some great plot twists	pos
It was pathetic. The worst part about it was the boxing scenes…	neg
…awesome caramel sauce and sweet toasty almonds. I love this place!	pos
…awful pizza and ridiculously overpriced…	neg

Classification

- Classification is one form of prediction, where the value to be predicted is a label.

Outlook	Temp (°F)	Humidity (%)	Windy	Class
sunny	75	70	true	play
sunny	80	90	true	don't play
sunny	85	85	false	don't play
sunny	72	95	false	don't play
sunny	69	70	false	play
overcast	72	90	true	play
overcast	83	78	false	play
overcast	64	65	true	play
overcast	81	75	false	play
rain	71	80	true	don't play
rain	65	70	true	don't play
rain	75	80	false	play
rain	68	80	false	play
rain	70	96	false	play

The goal of classification is to take a single observation, extract some useful features, and thereby classify the observation into one of a set of discrete classes.

Outlook=sunny	Temp=79	Humidity=88	Windy=false	Class=?

Text Classification

- Text classification is the task of assigning a label (categories, topics, or genres) to an entire text or document.
  - Spam detection
  - Authorship identification (who wrote this?)
  - Language Identification (is this Portuguese?)
  - Sentiment analysis (focus of this lecture)
  - …
- *Input*:
  - a document d
  - a fixed set of classes $C = \{c_1, c_2, \ldots, c_J\}$
- *Output*: a predicted class $c \in C$

Spam Detection

Is this spam or not?
Labels: “spam” and “not spam”
Binary classification task

Good morning Dan,

Please familiarize yourself with the attached file.
Reply here if you have any questions.

Thank you.

John and Mike,

Appreciate your flexibility this week, as the team navigates the sensitivities surrounding some of the project work taking place at the sites. Please tentatively plan for mobilization on 05/16/2022, in order to begin the final stages of the upgrade.

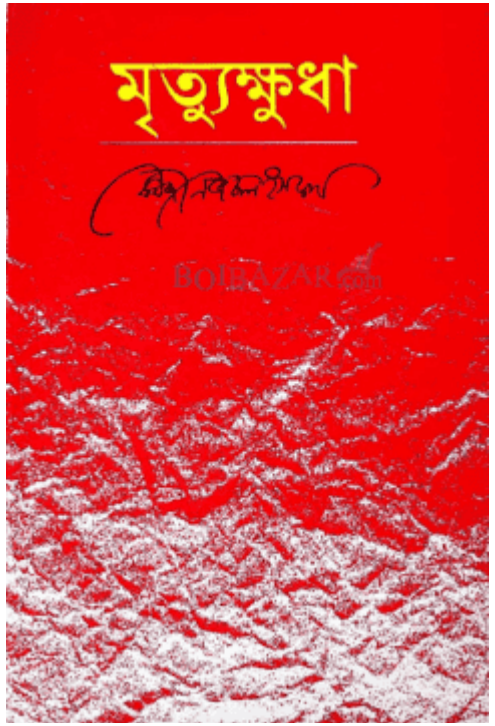
I will follow-up tomorrow with a confirmation if all indications are we will be given the “all-clear” before EOB Wednesday/SOB Thursday.

Appreciate your support.

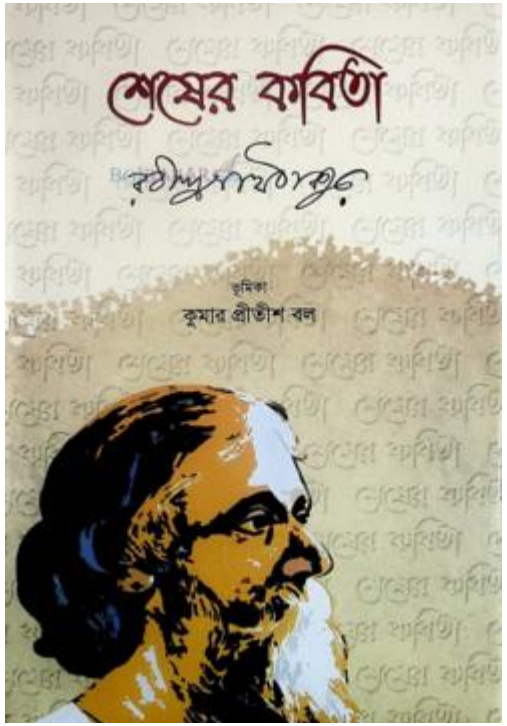
Regards,

Judy Sewell
Project Manager

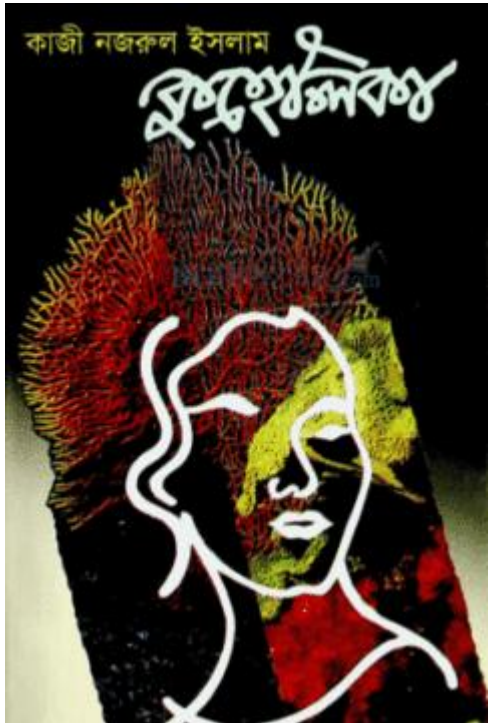
Authorship Attribution



Kazi Nazrul Islam



Rabindranath Tagore



???

Article Categorization

MEDLINE Article

Syntactic frame and verb bias in aphasia: Plausibility judgments of undergoer-subject sentences

?

MeSH Subject Category Hierarchy

- Antogonists and Inhibitors
- Blood Supply
- Chemistry
- Drug Therapy
- Embryology
- Epidemiology
- …

Sentiment Analysis

- The simplest version of sentiment analysis is a binary classification task, and the words of the review provide excellent cues.
- Consider, for example, the following phrases extracted from positive and negative reviews of movies and restaurants.
- Words like great, richly, awesome, and pathetic, and awful and ridiculously are very informative cues.

Text	Lebel
…zany characters and richly applied satire, and some great plot twists	pos
It was pathetic. The worst part about it was the boxing scenes…	neg
…awesome caramel sauce and sweet toasty almonds. I love this place!	pos
…awful pizza and ridiculously overpriced…	neg
This burger is pathetic.	???

Sentiment Analysis Methods

- A rule-based algorithm composed by a human (e.g., VADER scikit-learn)
- A machine learning model based on data (e.g., Naive Bayes scikit-learn)

Rule Based Approach

- Rules based on combinations of words or other features
  - spam: black-list-address OR (“dollars” AND “have been selected”)
- Accuracy can be high
  - In very specific domains
  - If rules are carefully refined by experts
- But:
  - rules can be fragile, as situations or data change over time
  - building and maintaining rules is expensive
  - they are too literal and specific: "high-precision, low-recall"

Machine Learning Approach

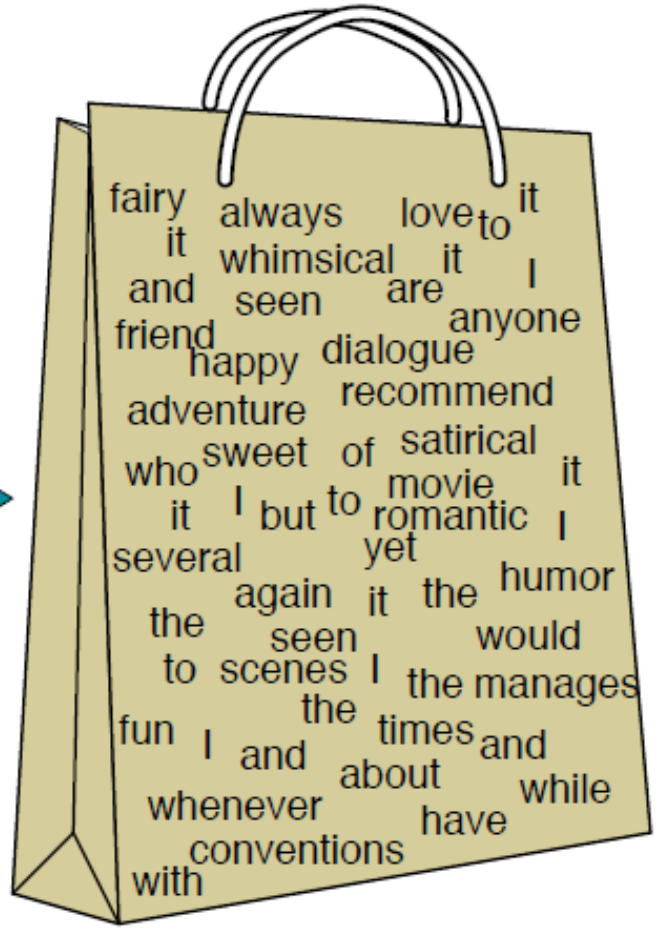
- Machine learning, relies on a labeled set of statements or documents to train a machine learning model to create rules.
- A machine learning sentiment model is trained to process input text and output a numerical value for the sentiment we are trying to measure. This approach is known as **supervised machine learning**.
- *Input:*
  - a document *d*
  - a fixed set of classes $C = \{c_1, c_2, \ldots, c_J\}$
  - A **training set** of *m* hand-labeled documents $(d_1,c_1),\ldots,(d_m,c_m)$
- *Output:*
  - a learned classifier $\gamma:d \rightarrow c$

Many kinds of classifiers!

- Naïve Bayes (this lecture)
- *k*-nearest neighbors
- Logistic regression
- Neural networks
- …

Bag of Words (BoW)

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

A **bag of words** is an unordered set of words whose position is ignored, keeping only their frequency in the document.

Naive Bayes Classifier

- This method is based on Bayes' rule.
- It relies on BoW representation.
- Naive Bayes is a probabilistic classifier, meaning that for a document d, out of all classes c $\in$ C the classifier returns the class c which has the maximum posterior probability given the document.

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

we want to predict the class $\hat{c}$

this is our test document d

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

- C = {c1, c2}
- c1 = "-"
- c2 = "+"

Anyone of these two will be $\hat{c}$

- d = "predictable with no fun"

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

$\hat{c}$ d

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

- C = {c1, c2}
- c1 = "-"
- c2 = "+"

Anyone of these two will be $\hat{c}$

- d = "predictable with no fun"

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

$\hat{c}$ d

Why conditional probability *P(c|d)*?

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

- C = {c1, c2}
- c1 = "-"
- c2 = "+"

Anyone of these two will be $\hat{c}$

- d = "predictable with no fun"

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

$\hat{c}$ d

Why *P(c|d)*? Because we want to find the probability value for (c1 & c2)
"if document d='predictable with no fun', then what is the value of class c"

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

- C = {c1, c2}
- c1 = "-"
- c2 = "+"
- d = "predictable with no fun"
- P(c1|d) = P("-" | "predictable with no fun")
- P(c2|d) = P("+" | "predictable with no fun")

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Why *P(c|d)*? Because we want to find the probability value for (c1 & c2)
"if document d='predictable with no fun', then what is the value of class c"

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

- C = {c1, c2}
- c1 = "-"
- c2 = "+"
- d = "predictable with no fun"
- P(c1|d) = P("-" | "predictable with no fun")
- P(c2|d) = P("+" | "predictable with no fun")

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Why *argmax*?

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

- C = {c1, c2}
- c1 = "-"
- c2 = "+"
- d = "predictable with no fun"
- P(c1|d) = P("-" | "predictable with no fun")
- P(c2|d) = P("+" | "predictable with no fun")

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Why *argmax*? Among multiple values, it selects the highest value. Our target is to select the class for which we obtain the highest probability.

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

- C = {c1, c2}
- c1 = "-"
- c2 = "+"
- d = "predictable with no fun"
- P(c1|d) = P("-" | "predictable with no fun")
- P(c2|d) = P("+" | "predictable with no fun")
- argmax [P("-" | "predictable with no fun"), P("+" | "predictable with no fun")] from this expression, select a class that has the highest probability.

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

$$P(x|y) = \frac{P(y|x)P(x)}{P(y)}$$

Bayes' rule

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

$$P(x|y) = \frac{P(y|x)P(x)}{P(y)}$$

Bayes' rule

$$P(c \mid d) = \frac{P(d \mid c)P(c)}{P(d)}$$

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

$$P(x|y) = \frac{P(y|x)P(x)}{P(y)}$$

Bayes' rule

$$P(c \mid d) = \frac{P(d \mid c)P(c)}{P(d)}$$

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d) = \underset{c \in C}{\operatorname{argmax}} \frac{P(d|c)P(c)}{P(d)}$$

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d) = \underset{c \in C}{\operatorname{argmax}} \frac{P(d|c)P(c)}{P(d)}$$

ignore denominator *P(d)*

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d) = \underset{c \in C}{\operatorname{argmax}} \frac{P(d|c)P(c)}{P(d)}$$

ignore denominator *P(d)*

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d) = \underset{c \in C}{\operatorname{argmax}} \frac{P(d|c)P(c)}{P(d)} = \underset{c \in C}{\operatorname{argmax}} P(d|c)P(c)$$

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d) = \underset{c \in C}{\operatorname{argmax}} \frac{P(d|c)P(c)}{P(d)}$$

ignore denominator *P(d)*

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c|d) = \underset{c \in C}{\operatorname{argmax}} \frac{P(d|c)P(c)}{P(d)} = \underset{c \in C}{\operatorname{argmax}} P(d|c)P(c)$$

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \overbrace{P(d|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}$$

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \overbrace{P(d|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}$$

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \overbrace{P(d|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}$$

A document *d* is represented as a set of features $f_1, f_2, \ldots, f_n$

$d = f_1, f_2, \ldots, f_n$

d = "predictable with no fun"

"predictable with no fun" = {"predictable", "with", "no", "fun"}

f_1 = "predictable", f_2 = "with", f_3 = "no", f_4 = "fun"
hence, **$d = \{f_1, f_2, f_3, f_4\}$**

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \overbrace{P(d|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}$$

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \overbrace{P(f_1, f_2, ..., f_n|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}$$

A document *d* is represented as a set of features $f_1, f_2, \ldots, f_n$

$d = f_1, f_2, \ldots, f_n$

d = "predictable with no fun"

"predictable with no fun" = {"predictable", "with", "no", "fun"}

f_1 = "predictable", f_2 = "with", f_3 = "no", f_4 = "fun"

hence, **$d = \{f_1, f_2, f_3, f_4\}$**

Naive Bayes Classifier

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \overbrace{P(d|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}$$

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \overbrace{P(f_1, f_2, ..., f_n|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}$$

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c)\ P(f_1, f_2, ..., f_n|c)$$

A document *d* is represented as a set of features $f_1, f_2, \ldots, f_n$

$d = f_1, f_2, \ldots, f_n$

d = "predictable with no fun"

"predictable with no fun" = {"predictable", "with", "no", "fun"}

f_1 = "predictable", f_2 = "with", f_3 = "no", f_4 = "fun"
hence, **$d = \{f_1, f_2, f_3, f_4\}$**

Naive Bayes Classifier (Assumptions)

- To calculate $P(f_1, f_2, \ldots, f_n|c)$, Naive Bayes classifiers make two simplifying assumptions.

Naive Bayes Classifier (Assumptions)

- To calculate $P(f_1, f_2, \ldots, f_n|c)$, Naive Bayes classifiers make two simplifying assumptions.
  - **Bag of Words assumption**: we assume feature position doesn't matter

Naive Bayes Classifier (Assumptions)

- To calculate $P(f_1, f_2, \ldots, f_n|c)$, Naive Bayes classifiers make two simplifying assumptions.
  - **Bag of Words assumption**: we assume feature position doesn't matter
  - **naive Bayes assumption**: this is the conditional independence assumption that the probabilities $P(f_i|c)$ are independent given the class c.

Naive Bayes Classifier (Assumptions)

- To calculate $P(f_1, f_2, \ldots, f_n|c)$, Naive Bayes classifiers make two simplifying assumptions.
  - **Bag of Words assumption**: we assume feature position doesn't matter
  - **naive Bayes assumption**: this is the conditional independence assumption that the probabilities $P(f_i|c)$ are independent given the class c.

$$P(f_1, f_2, \ldots, f_n|c) = P(f_1|c) \cdot P(f_2|c) \cdot \ldots \cdot P(f_n|c) = \prod_{f \in F} P(f|c)$$

Naive Bayes Classifier (Assumptions)

- To calculate $P(f_1, f_2, \ldots, f_n|c)$, Naive Bayes classifiers make two simplifying assumptions.
  - **Bag of Words assumption**: we assume feature position doesn't matter
  - **naive Bayes assumption**: this is the conditional independence assumption that the probabilities $P(f_i|c)$ are independent given the class c.

$$P(f_1, f_2, \ldots, f_n|c) = P(f_1|c) \cdot P(f_2|c) \cdot \ldots \cdot P(f_n|c) = \prod_{f \in F} P(f|c)$$

$$P(f_1, f_2, \ldots, f_n|c) = \prod_{f \in F} P(f|c)$$

Naive Bayes Classifier

- Let's back to our equation simplification

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}}\, P(c)\; P(f_1, f_2, \ldots., f_n | c)$$

Naive Bayes Classifier

- Let's back to our equation simplification

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}}\, P(c)\, P(f_1, f_2, \ldots., f_n|c)$$

$$P(f_1, f_2, \ldots., f_n|c) = \prod_{f \in F} P(f|c)$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}}\, P(c) \prod_{f \in F} P(f|c)$$

Naive Bayes Classifier

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{f \in F} P(f|c)$$

- To apply the naive Bayes classifier to text, **we will use each word in the documents as a feature**, and we consider each of the words in the document by walking an index through every word position in the document.

Naive Bayes Classifier $c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{f \in F} P(f|c)$

- To apply the naive Bayes classifier to text, **we will use each word in the documents as a feature**, and we consider each of the words in the document by walking an index through every word position in the document.

$$\text{positions} \leftarrow \text{all word positions in test document}$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i|c)$$

d = "predictable with no fun"
d = {w_1, w_2, w_3, w_4}
w_1 = "predictable", w_2 = "with", w_3 = "no", w_4 = "fun"

Naive Bayes Classifier

- To apply the naive Bayes classifier to text, **we will use each word in the documents as a feature**, and we consider each of the words in the document by walking an index through every word position in the document.

$$\text{positions} \leftarrow \text{all word positions in test document}$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i|c)$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} \log P(c) + \sum_{i \in positions} \log P(w_i|c)$$

Naive Bayes calculations, like calculations for language modeling, are done in log space, to avoid underflow and increase speed.

Naive Bayes Classifier

- Now we have the following simplified equation. We need to calculate *P(c)* and $P(w_i|c)$ to make our prediction.

This is the simplification of *P(c|d)*

$$positions \leftarrow \text{all word positions in test document}$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i|c)$$

P("-" | "predictable with no fun") = P("-")* P("predictable"|"-")*P("with"|"-")*P("no"|"-")*P("fun"|"-")

argmax[P("-" | "predictable with no fun"), P("+" | "predictable with no fun")]
argmax[P("-")* P("predictable"|"-")*P("with"|"-")*P("no"|"-")*P("fun"|"-"),
P("+")* P("predictable"|"+")*P("with"|"+")*P("no"|"+")*P("fun"|"+")]

Naive Bayes Classifier

- Learn *P(c)*. Let N_c be the number of documents in our training data with class c and N_{doc} be the total number of documents.

$$\hat{P}(c) = \frac{N_c}{N_{doc}}$$

Naive Bayes Classifier

- Learn *P(c)*. Let N_c be the number of documents in our training data with class c and N_{doc} be the total number of documents.

$$\hat{P}(c) = \frac{N_c}{N_{doc}}$$

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Naive Bayes Classifier

- Learn *P(c)*. Let N_c be the number of documents in our training data with class c and N_{doc} be the total number of documents.

$$\hat{P}(c) = \frac{N_c}{N_{doc}}$$

- P(+) = 2/5
- P(-) = 3/5

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Naive Bayes Classifier

- Learn $P(w_i|c)$

$$\hat{P}(w_i|c) = \frac{count(w_i, c)}{\sum_{w \in V} count(w, c)}$$

Naive Bayes Classifier

- Learn $P(w_i|c)$

$$\hat{P}(w_i|c) = \frac{count(w_i, c)}{\sum_{w \in V} count(w, c)}$$

The vocabulary V consists of the union of all the word types in all classes, not just the words in one class c.

Naive Bayes Classifier

- Learn $P(w_i|c)$

$$\hat{P}(w_i|c) = \frac{count(w_i, c)}{\sum_{w \in V} count(w, c)}$$

The vocabulary V consists of the union of all the word types in all classes, not just the words in one class c.

- P("predictable"|-) = ?

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Naive Bayes Classifier

- Learn $P(w_i|c)$

$$\hat{P}(w_i|c) = \frac{count(w_i, c)}{\sum_{w \in V} count(w, c)}$$

The vocabulary V consists of the union of all the word types in all classes, not just the words in one class c.

- P("predictable"|-) = 1/14

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Naive Bayes Classifier

- Learn $P(w_i|c)$

$$\hat{P}(w_i|c) = \frac{count(w_i, c)}{\sum_{w \in V} count(w, c)}$$

The vocabulary V consists of the union of all the word types in all classes, not just the words in one class c.

- P("predictable"|-) = 1/14
- P("fun"|-) = 0/14 = 0

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Naive Bayes Classifier

- Learn $P(w_i|c)$

$$\hat{P}(w_i|c) = \frac{count(w_i, c)}{\sum_{w \in V} count(w, c)}$$

The vocabulary V consists of the union of all the word types in all classes, not just the words in one class c.

$$\hat{P}(w_i|c) = \frac{count(w_i, c) + 1}{\sum_{w \in V} (count(w, c) + 1)} = \frac{count(w_i, c) + 1}{\left(\sum_{w \in V} count(w, c)\right) + |V|}$$

Laplace smoothing

Naive Bayes Classifier

$$\hat{P}(w_i|c) = \frac{count(w_i,c)+1}{\sum_{w\in V}(count(w,c)+1)} = \frac{count(w_i,c)+1}{\left(\sum_{w\in V} count(w,c)\right)+|V|}$$

- P("predictable"|-) = (1+1)/(14+20)
- P("no"|-) = (1+1)/(14+20)
- P("fun"|-) = (0+1)/(14+20)

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Naive Bayes Classifier

$$\hat{P}(w_i|c) = \frac{count(w_i,c)+1}{\sum_{w\in V}(count(w,c)+1)} = \frac{count(w_i,c)+1}{\left(\sum_{w\in V} count(w,c)\right)+|V|}$$

- P("predictable"|-) = (1+1)/(14+20)
- P("no"|-) = (1+1)/(14+20)
- P("fun"|-) = (0+1)/(14+20)

- P("predictable"|+) = (0+1)/(9+20)
- P("no"|+) = (0+1)/(9+20)
- P("fun"|+) = (1+1)/(9+20)

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Naive Bayes Classifier

$$\text{positions} \leftarrow \text{all word positions in test document}$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i|c)$$

argmax[P("-" | "predictable with no fun"), P("+" | "predictable with no fun")]

argmax[P("-")* P("predictable"|"-")*P("with"|"-")*P("no"|"-")*P("fun"|"-"),
P("+")* P("predictable"|"+")*P("with"|"+")*P("no"|"+")*P("fun"|"+")]

Naive Bayes Classifier

$$\text{positions} \leftarrow \text{all word positions in test document}$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i|c)$$

argmax[P("-" | "predictable with no fun"), P("+" | "predictable with no fun")]

argmax[P("-")* P("predictable"|"-")*P("with"|"-")*P("no"|"-")*P("fun"|"-"),
P("+")* P("predictable"|"+")*P("with"|"+")*P("no"|"+")*P("fun"|"+")]

- P(-|"predictable with no fun") = P(-)* P("predictable"|-)*P("with"|-)
*P("no"|-)*P("fun"|-)

Naive Bayes Classifier

$$\text{positions} \leftarrow \text{all word positions in test document}$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i|c)$$

argmax[P("-" | "predictable with no fun"), P("+" | "predictable with no fun")]

argmax[P("-")* P("predictable"|"-")*P("with"|"-")*P("no"|"-")*P("fun"|"-"),
P("+")* P("predictable"|"+")*P("with"|"+")*P("no"|"+")*P("fun"|"+")]

- P(-|"predictable with no fun") = P(-)* P("predictable"|-)*P("with"|-) *P("no"|-)*P("fun"|-) = (3/5) * (2/34) * (2/34) * (1/34)

Naive Bayes Classifier

$$\text{positions} \leftarrow \text{all word positions in test document}$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i|c)$$

argmax[P("-" | "predictable with no fun"), P("+" | "predictable with no fun")]

argmax[P("-")* P("predictable"|"-")*P("with"|"-")*P("no"|"-")*P("fun"|"-"),
P("+")* P("predictable"|"+")*P("with"|"+")*P("no"|"+")*P("fun"|"+")]

- P(-|"predictable with no fun") = P(-)* P("predictable"|-)*P("with"|-) *P("no"|-)*P("fun"|-) = (3/5) * (2/34) * (2/34) * (1/34) = 6.1 x 10^{-5}

Naive Bayes Classifier

$$\text{positions} \leftarrow \text{all word positions in test document}$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i|c)$$

argmax[P("-" | "predictable with no fun"), P("+" | "predictable with no fun")]

argmax[P("-")* P("predictable"|"-")*P("with"|"-")*P("no"|"-")*P("fun"|"-"),
P("+")* P("predictable"|"+")*P("with"|"+")*P("no"|"+")*P("fun"|"+")]

- P(-|"predictable with no fun") = P(-)* P("predictable"|-)*P("with"|-) *P("no"|-)*P("fun"|-) = (3/5) * (2/34) * (2/34) * (1/34) = 6.1 x 10^{-5}
- P(+|"predictable with no fun") = P(+)* P("predictable"|+)*P("with"|+) *P("no"|+)*P("fun"|+) = (2/5) * (1/329) * (1/29) * (2/29) = 3.2 x 10^{-5}

Naive Bayes Classifier

$$\text{positions} \leftarrow \text{all word positions in test document}$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i|c)$$

argmax[P("-" | "predictable with no fun"), P("+" | "predictable with no fun")]

argmax[P("-")* P("predictable"|"-")*P("with"|"-")*P("no"|"-")*P("fun"|"-"),
P("+")* P("predictable"|"+")*P("with"|"+")*P("no"|"+")*P("fun"|"+")]

- P(-|"predictable with no fun") = 6.1 x 10^{-5}
- P(+|"predictable with no fun") = 3.2 x 10^{-5}

Naive Bayes Classifier

$$\text{positions} \leftarrow \text{all word positions in test document}$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i|c)$$

argmax[P("-" | "predictable with no fun"), P("+" | "predictable with no fun")]

argmax[P("-")* P("predictable"|"-")*P("with"|"-")*P("no"|"-")*P("fun"|"-"),
P("+")* P("predictable"|"+")*P("with"|"+")*P("no"|"+")*P("fun"|"+")]

- P(-|"predictable with no fun") = 6.1 x 10^{-5}
- P(+|"predictable with no fun") = 3.2 x 10^{-5}

argmax[P("-" | "predictable with no fun"), P("+" | "predictable with no fun")]
= "-"

Naive Bayes Classifier

- **Unknown Words**: What do we do about words that occur in our test data but are not in our vocabulary at all because they did not occur in any training document in any class? **We ignore them—remove them from the test document.**
- **Stop Words**: Some systems ignore stop words – remove them. In practice, most NB algorithms use all words and don't use stopword lists.

Optimizing for Sentiment Analysis

- for sentiment classification and a number of other text classification tasks, whether a word occurs or not seems to matter more than its frequency.
- Thus it often improves performance to clip the word counts in each document at 1.
- This variant is called binary multinomial naive Bayes or binary naive Bayes.

Optimizing for Sentiment Analysis

Four original documents:

- − it was pathetic the worst part was the boxing scenes
- − no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

After per-document binarization:

- − it was pathetic the worst part boxing scenes
- − no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

	NB Counts		Binary Counts	
	+	−	+	−
and	2	0	1	0
boxing	0	1	0	1
film	1	0	1	0
great	3	1	2	1
it	0	1	0	1
no	0	1	0	1
or	0	1	0	1
part	0	1	0	1
pathetic	0	1	0	1
plot	1	1	1	1
satire	1	0	1	0
scenes	1	2	1	2
the	0	2	0	1
twists	1	1	1	1
was	0	2	0	1
worst	0	1	0	1

Figure 4.3 An example of binarization for the binary naive Bayes algorithm.

Optimizing for Sentiment Analysis

- An important addition commonly made when doing text classification for sentiment is to deal with negation.
- Consider the difference between "I really like this movie" (positive) and "I didn't like this movie" (negative).
- The negation expressed by didn't completely alters the inferences we draw from the predicate "like."

Optimizing for Sentiment Analysis

- A very simple baseline that is commonly used in sentiment analysis to deal with negation is the following: during text normalization, prepend the prefix *NOT_* to every word after a token of logical negation (*n't, not, no, never*) until the next punctuation mark.

```
didn't like this movie , but I
```

becomes

```
didn't NOT_like NOT_this NOT_movie , but I
```

Optimizing for Sentiment Analysis

- In some situations we might have insufficient labeled training data to train accurate naive Bayes classifiers using all words in the training set to estimate positive and negative sentiment.
- In such cases we can instead derive the positive and negative word features from sentiment lexicons, lists of words that are pre-annotated with positive or negative sentiment.
- Four popular lexicons are the General Inquirer (Stone et al., 1966), LIWC (Pennebaker et al., 2007), the opinion lexicon Hu and Liu (2004a) and the MPQA Subjectivity Lexicon (Wilson et al., 2005).
- For example the MPQA subjectivity lexicon has 6885 words each marked for whether it is strongly or weakly biased positive or negative. Some examples:

  \+ : *admirable, beautiful, confident, dazzling, ecstatic, favor, glee, great*

  − : *awful, bad, bias, catastrophe, cheat, deny, envious, foul, harsh, hate*

Evaluation: Accuracy, Precision, Recall, F-measure

- Let's first address binary classifiers:
  - Is this email spam?

    spam (+) or not spam (-)
  - Is this post about Delicious Pie Company?

    about Del. Pie Co (+) or not about Del. Pie Co(-)
- For each item (email document) we need to know whether our system called it spam or not.
- We also need to know whether the email is actually spam or not, i.e. the human-defined labels for each document that we are trying to match.
- We refer to human labels as the gold labels.

Confusion Matrix

- To evaluate any system for detecting things, we start by building a confusion matrix.
- A confusion matrix is a table for visualizing how an algorithm performs with respect to the human gold labels, using two dimensions (system output and gold labels), and each cell labeling a set of possible outcomes.

		gold standard labels	
		gold positive	gold negative
system output labels	system positive	**true positive**	**false positive**
	system negative	**false negative**	**true negative**

Accuracy

		gold standard labels	
		gold positive	gold negative
system output labels	system positive	**true positive**	**false positive**
	system negative	**false negative**	**true negative**

$$\textbf{accuracy} = \frac{tp+tn}{tp+fp+tn+fn}$$

Precision & Recall

Accuracy doesn't work well when we're dealing with uncommon or imbalanced classes

Suppose we look at 1,000,000 social media posts to find Delicious Pie-lovers (or haters)

- 100 of them talk about our pie
- 999,900 are posts about something unrelated

Imagine the following simple classifier

Every post is "not about pie"

Precision & Recall

Accuracy of our "nothing is pie" classifier

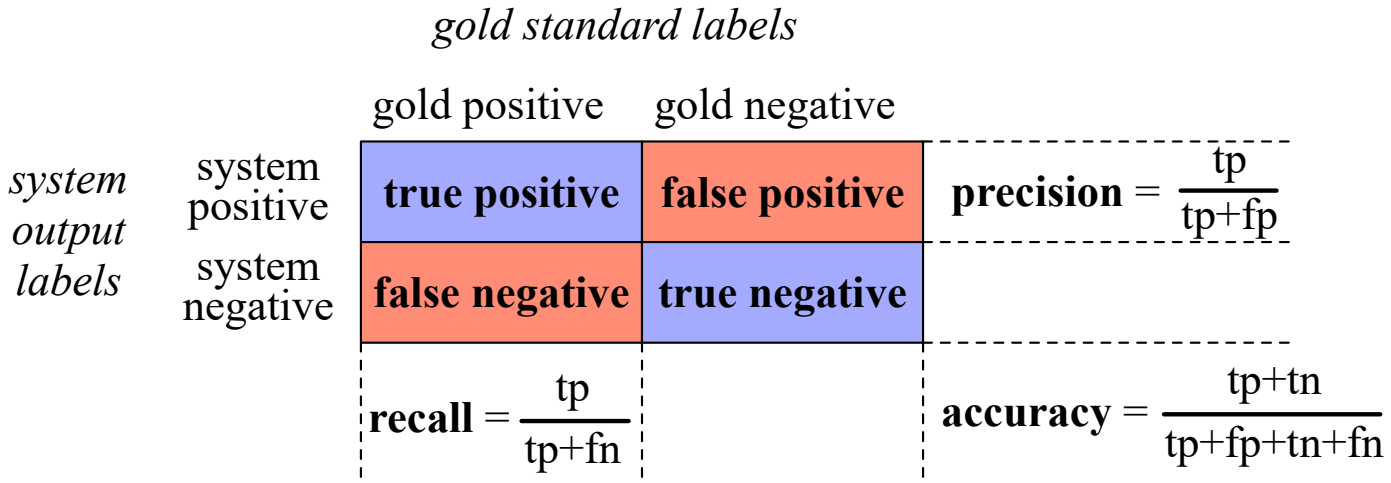
- 999,900 true negatives and 100 false negatives
- Accuracy is 999,900/1,000,000 = 99.99%!
- But useless at finding pie-lovers (or haters)!!
- Which was our goal!

Accuracy doesn't work well for unbalanced classes

- Most tweets are not about pie!

Precision & Recall

- Precision: % of selected items that are correct
  - Precision measures the percentage of the items that the system detected (i.e., the system labeled as positive) that are in fact positive (i.e., are positive according to the human gold labels).
- Recall: % of correct items that are selected
  - Recall measures the percentage of items actually present in the input that were correctly identified by the system.



Precision & Recall

Stupid classifier: Just say no: every tweet is "not about pie"

- 100 tweets talk about pie, 999,900 tweets don't
- Accuracy = 999,900/1,000,000 = 99.99%

But the Recall and Precision for this classifier are terrible:

$$\textbf{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

$$\textbf{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

F1- Score

- F1 is a combination of precision and recall.

$$F_1 = \frac{2PR}{P+R}$$

References

- Daniel Jurafsky and James H. Martin. Speech and Language Processing, 3rd edition, 2024.
- Hobson Lane and Maria Dyshel. Natural Language Processing in Action, 2nd Edition, 2025.
- https://www.analyticsvidhya.com/blog/2022/03/building-naive-bayes-classifier-from-scratch-to-perform-sentiment-analysis/
- https://www.youtube.com/watch?v=YVC-L_lLQb4